MBRL agent

Baseline alg:

```
Algorithm 1 Model-based Reinforcement Learning
1: gather dataset $\mathcal{D}_{RAND}$ of random trajectories
2: initialize empty dataset $\mathcal{D}_{RL}$, and randomly initialize $\hat{f}_\theta$
3: for iter=1 to max_iter do
4:     train $\hat{f}_\theta(s, a)$ by performing gradient descent on Eqn. 2,
       using $\mathcal{D}_{RAND}$ and $\mathcal{D}_{RL}$
5:     for $t = 1$ to $T$ do
6:         get agent's current state $s_t$
7:         use $\hat{f}_\theta$ to estimate optimal action sequence $A_t^{(H)}$ (Eqn. 4)   ← CEM
8:         execute first action $a_t$ from selected action sequence $A_t^{(H)}$   ← MPC step
9:         add $(s_t, a_t)$ to $\mathcal{D}_{RL}$
10:    end for
11: end for
```

could initialize via human controlled rollouts to ensure small distribution shift?

D_{RL}: buffer of transitions gathered via policy

D_{RAND}: transitions via initial random policy

they're just used so could combine

check CS285 HW 4 2023 for implementation tips → $\Delta_s = f(s_t, a_t)$ rather than s_{t+1} … why? → normalization?

code architecture: see CS285 for inspo

used in CS285 just to 'update the statistics used to normalize the inputs & outputs of the dynamics model'

dynamics model:
- update: uses a batch of transitions, take one NN optimization step — normalized?
- update_statistics: used for normalization?
- get_dynamics_prediction: $o_t, a_t \to o_{t+1}$ (ens) uses one of the dynamics models

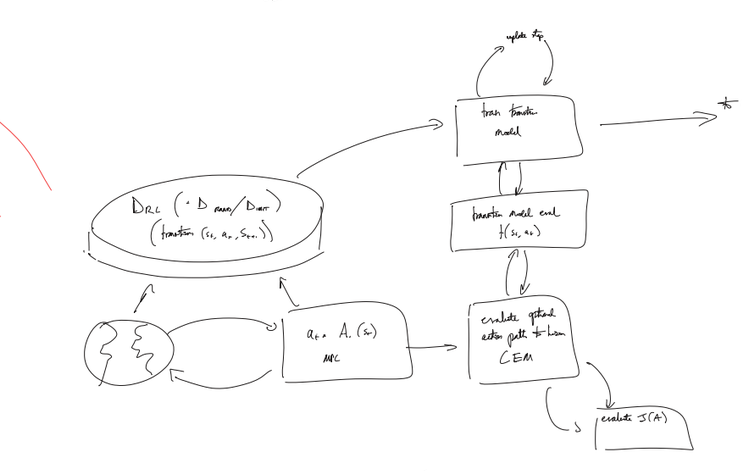
MPC:
- evaluate_action_sequence: $A \to J(A)$ across ensemble of dynamics model
- action selection: run MPC alg to get $s_t \to A$ ← CEM

uses facebook library structure, defines the transition batch structure

defined as a class: function are defined inside an self

replay buffer:
- sample: take a batch of given length out of buffer
- insert: insert a single transition into buffer
- batched_insert: insert a batch of transitions into buffer
- init to define class?

defined with a size: once exceeded …


update step
train transition model
transition model eval $f(s_t, a_t)$
evaluate optimal action path to horizon CEM
evaluate J(A)
D_{RL} (+ D_{rand}/D_{init}) (transitions (s_t, a_t, s_{t+1}))
$a_t = A_t(s_t)$ MPC

→ output: save parameters

(→ could use DataParallel for faster …)

build architecture for various lengths & transition models for multitask learning

means to initialize a model free agent → see paper (essentially just imitation learning) "expert trajectories"

… for model?

can use this as the basis for DAGGER

Implementation: · assuming continuous action

Transition model:

Model as DT or Δt: ← can choose h_t: longer helps for longer MPC paths & helps get 'more information from each datapoint' but prediction/modelling over long Δt can be hard (underlying dynamics are continuous)

+ deterministic model

$s_{t+1} = f_\theta(s_t, a_t)$ ✗

↑ hard if states have small change

⇓

$s_{t+1} = s_t + f_\theta(s_t, a_t)$

$f_\theta(s_t, a_t) = \Delta s$ or Δt

DT model: how does this relate to env? ?? sublevel as underlying continuous or not?

train on data $x_t = (s_t, a_t)$, $y_t = (s_{t+1} - s_t)$ + data processing: with Δs_t, there are terms of very different magnitudes ↳ to make the loss function treat them equally we have to normalize (each term): $y_t = \frac{\Delta s_t - \overline{\Delta s}}{\sigma_{\Delta s}} \approx f_\theta(s_t, a_t)$

?? : look into more

+ they added Gaussian noise to y_t to decrease robustness ← for simulator error allowance or just to plan better under (and model uncertainty) with noise?

⇓

$D = \{ ((s_t, a_t), y_t) \dots \}$ → in python: in replay buffer

this impacts model evaluation now:

$\Delta s_t = \sigma_{\Delta s} f_\theta(s_t, a_t) + \overline{\Delta s}$

from the D used to train f_θ ↳ then change over time? The latest estimate?

Training:

$$\mathcal{L}(\theta) = \frac{1}{|D|} \sum_{(s_t, a_t, s_{t+1}) \in D} \frac{1}{2} \| y_t - f_\theta(s_t, a_t) \|^2$$

optimization: SGD

calculate on both D_{train} & D_{val}

paper mentioned Adam? CHECK

extended to capture open loop patterns to limit trajectories in Berkeley paper (H-step loss)

more useful for analysis than training

could train off of this?

ensemble: ways to handle model uncertainty, number? ↳ pick & choose which to approximate $p(\theta | D)$ → easiest is to initialize them independently & train on the same data, but could also train on different data

↳ sample from replay buffer based off of different random seeds?

↳ + implement: ensemble MPC

average? min?

doesn't calculate expectation via samples but doesn't nothing to avoid paths of high uncertainty? how to do this?

Buffer: · uses facebook/Berkeley structure
· need D_{train}, D_{val}: split fully separate?

need to add $\overline{\Delta s}$ & $\sigma_{\Delta s}$ calls, plausibility (+ faster)

MPC: for open loop planning & 'closed loop' action selection

2 key concepts: value & optimization

Value: calculate just expected reward: we need known env reward function, learn it or define it

deterministic dynamics: $J(A) = \sum_{t=1}^{H} r(s_t, a_t)$ where $s_{t+1} = f(s_t, a_t)$

stochastic dynamics: $J(A) = \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{H} r(s_{t,i}, a_t)$, where $s_{t+1,i} = f_{\theta_i}(s_{t,i}, a_t)$

Using bootstrapping ensemble: simple baseline algorithm and effective stochastic models

stochastic env of candidate J is the avg:

In general, for candidate action sequence $a_1, \dots, a_H$:
- Step 1: sample $\theta \sim p(\theta|\mathcal{D})$
- Step 2: at each time step t, sample $s_{t+1} \sim p(s_{t+1}|s_t, a_t, \theta)$
- Step 3: calculate $R = \sum_t r(s_t, a_t)$
- Step 4: repeat steps 1 to 3 and accumulate the average reward

sample A offline to learned model

↳ use the same model θ_i for each trajectory for physical consistency

the often debated: …

is this same way to add a cost for model disagreement? capture extrapolation?

Optimization

① CEM

iterative guess & check, focuses in on optimal region

cross-entropy method with continuous-valued inputs:
1. sample $A_1, \dots, A_N$ from $p(A)$
2. evaluate $J(A_1), \dots, J(A_N)$
3. pick the elites $A_{i_1}, \dots, A_{i_M}$ with the highest value, where $M < N$ → pick the best M A_i
4. refit $p(A)$ to the elites $A_{i_1}, \dots, A_{i_M}$

refit the A samples distribution around the elites to make them more likely

does each action get sampled individually or the whole path?? for a d-step A: A is a d-dimensional RV ↳ in practice each element each normally distributed $p(A) = \prod_t N(a_t; \mu_t, \sigma_t^2)$

benefits:
- fast if J(A) evaluations are parallelized
- simple

issues: (key points)
- hard dimensionality but on number of actions
- only considers J(A): value of whole path ↳ can't discriminate which action within that was good or bad
- struggles with multimodal path options ↳ averaging/refitting gaussians that span different modes ends up in the middle

this can't multimodal path options

optimization conditioned from initial state but not from initial action ↳ throughout optimization we assume actuator can change value at ∞ rate

improvements:
- J(A) to avoid ? σ model
- iCEM
- Decent CEM

Implementation:

$A = (a_0, a_1, \dots, a_{H-1})$

each a_i = (n_a) → $a_{ij} \sim N(\mu_{ij}, \sigma_{ij}^2)$

would mean a total of $H \times n_a$ distributions stored & $H \times n_a$ Continuous Random Variables ↳ defined via a mean vector & covariance matrix ← would be diagonal in this case

to initialize: $\mu_{ij} = \frac{a_{max} - a_{min}}{2}$, σ

code implementation: for parallel pytorch tensors → wrapped to be from pytorch every time would be a conversion burden ↓ run CEM in pytorch tensors ↳ pytorch multivariate normal: $A = (a_{11}, \dots, a_{Hn})$, $\mu = (\mu_{11}, \dots, \mu_{Hn})$, $\sigma = \mathrm{diag}(\sigma_{11}, \dots, \sigma_{Hn})$

can use .reshape to get into form: H × n_a (sampled … (averaged by J(A)))

trajectory_stack: ((A_1), (A_2), …)

reward_stack: (J(A_1), J(A_2), …)

Algorithm

```
Algorithm 1 Model-based Reinforcement Learning
1: gather dataset $\mathcal{D}_{RAND}$ of random trajectories
2: initialize empty dataset $\mathcal{D}_{RL}$, and randomly initialize $\hat{f}_\theta$
3: for iter=1 to max_iter do
4:     train $\hat{f}_\theta(s, a)$ by performing gradient descent on Eqn. 2,
       using $\mathcal{D}_{RAND}$ and $\mathcal{D}_{RL}$
5:     for $t = 1$ to $T$ do
6:         get agent's current state $s_t$
7:         use $\hat{f}_\theta$ to estimate optimal action sequence $A_t^{(H)}$ (Eqn. 4)
8:         execute first action $a_t$ from selected action sequence $A_t^{(H)}$
9:         add $(s_t, a_t)$ to $\mathcal{D}_{RL}$
10:    end for
11: end for
```

'on policy data aggregation'

results from paper:

Figure 6: approximate options:
- D_{rand}/D_{RL}: low D_{rand} is better, ~10% or less
- H/K: MPC horizon H, Sample at each time step K ← if too low can be greedy & get stuck in unrecoverable states; too high: model prediction breaks off of unreliable extrapolation

but this was for random shooting not CEM

H=10, K=1000 fixed in paper

- 60 'epochs' per aggregation iteration (but 200 is de facto, speed tradeoffs)
- rollouts: 400 (higher is better)

gradient descent steps — resample D each time?

real environment rollouts

how many CEM iterations per step?

Normalization Overview:

↳ a few different options: → 1. 2017 paper: μ & σ are defined once on DRAND & locked (~10k transitions)

more modern approach: recalculate each aggregation loop

- both x & y are normalized → $x_{normed} = \frac{x - \mu_x}{\sigma_x}$, $y_{normed} = \frac{y - \mu_y}{\sigma_y}$ ← for each dimension

(s_t, a_t): better for optimization; Δs_t: allows for more equal weights of errors

$\mu_x, \sigma_x, \mu_y, \sigma_y$: all formed …

- for the model: un-normalize

$s_{t+1} = s_t + \Delta_s$

$s_{t+1} = s_t + f_\theta \sigma_y + \mu_y$

Env

Hopper …

Debugging / Tests

① Dimension mismatch:

A … (H, n_a, 0)

A sample output: n_samples × MPC horizon × n_a ✓

forward pass: A_test: 1 × 10 × 10 = 6; 5 1× traj_n × H × n_a ✓

② Transition model loss not decreasing: ↳ loss plateau at 1.0

$\mathcal{L} = \sum \|y - f(x)\|^2$; $y = \frac{\Delta s - \mu}{\sigma}$

even with sample size = 5G, $\mathcal{L}$ plateau at 1.0 : not learning

↳ recheck shapes, optimization

x_t, y_t ✓

→ other possible issues:
- data x & y not matching → unlikely based off samples …
- normalization issues, dimension → (sample, n_o) − (1, n_o) ↳ gets broadcasted to (sample, n_o), then elementwise
- Adam optimizer? → now learning?? why did SGD not work??

③ Learns to flip

lengthen MPC horizon? ↳ 0.5s? 1s? ↑ flips

if it learns flip physics well accurately, models agree …, i.e. reward looks better?

but increase risk of planning based off of large model extrapolation

⇓

maybe implement CEM with: for a A_i: multiplying reward

i.e. for a given path, have some reward discounting but longer H

once flipped can't recover? & env doesn't reset? ↳ maybe force reset every ~5s: steps = T/Δt = 5/0.01 = 500? 100 seemed to be enough for it to flip & get stuck ('50')

↳ deleted flip & reset?

→ currently reset every 200/100 because of flipping anyway ↳ that's fine

cost fxn:

↳ transition model loss 0.02 → 0.01 every … iterations, good?

to stop plateau:
- more CEM candidates
- penalize z → implemented
- check CEM parameters

- model caching
- MPC multistep rollouts? → would allow for more transitions needed for same compute: each action with frame skip = 5 × 0.01 = 0.05s already → implemented as a config variable

- model trains on the same transition multiple times if SGD steps > available transitions → add a no-repeats in our training loop?
- random model choice for path eval, maybe average → implemented

Maybe split a light model for learning CEM & run with a heavier one physics?

→ Speed: vectorising? → pass all candidates to model at once: pytorch model runs on each row independently, with each row as traj_n

before, one action path: A: H × n_a

↳ now: A_batch: N × H × n_a (traj_n) samples, averaged and …

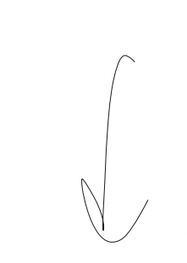
· make only one call to expected reward: don't loop over trajectories ↳ changes to expected_MPC_reward: stores: (traj_n,)

· transition model now handles states: traj_n × n_o, pytorch does this automatically

how good running demo!!

↳ what's next:
① Learning improvement:
② Analysis

① Improvement
- discounted CEM reward? test with penalty to term
- trend:



plateau as Δloss over training → 0 ← distribution shift?

↑ final loss ~ 0.012?

conclusion to be made about optimal H?

Loss of plasticity? Make model wider & deeper?

this is what was in paper → by ? loss doesn't actually go down

· saving & loading model

② Analysis ↓ paper comparison: → physics model for long profiles; tracking curves → implementation; compare reward

new goal: save physics model and on runtime can choose MPC reward & see visually

↳ to save post training:
- ensemble model weights
- normalization stats
- maybe config? i.e. can it change down the line?

→ running straight: 5 < x speed < 8; x speed ~ 6.5 m s⁻¹

velocity tests: reward = −|velocity error| …

others: gradual run: 4 aggregation, ~~60%~~ 50% train, ~~40%~~ 50% Drand (can be done more, fast enough, same as at max 5.5, check sim diff) + 4 × 300 (velocity reward)

after to get both of these tasks = all tasks ← run Drand